

7.6 研究日志整理版

light_cci + Laplacian-HKS + Sparse Unbalanced OT

目标：用低成本 CCI/DDI 图结构产生跨时间 PIJ，并避免大规模 expression/CCI mix 特征导致的计算爆炸。

核心边界。本方案的主结果只使用每个层级自身的 light_cci 图、Laplacian-HKS 特征、SR/发育状态量、空间位置，以及跨时间候选边。主 PIJ 不使用 lower-to-upper overlap、domain 映射或跨层标签作为成本或候选边来源，避免把 upper 层结构提前泄露给 lower 层。

Contents

1 总体思路	3
2 符号系统	3
2.1 样本组与节点集合	3
2.2 LR pair 与 COMMOT 通讯矩阵	3
3 Laplacian-HKS 节点特征	4
3.1 从 light_cci 邻接矩阵到 normalized Laplacian	4
3.2 谱分解与 HKS	4
4 source-target 预成本与候选集合	4
4.1 两个时间点的图与节点特征	5
4.2 预成本 B_{ij}	5
4.3 候选集合 Ω	5
5 辅助图与路径成本	6
5.1 辅助图定义	6
5.2 内部边代价	6
5.3 最终 OT 成本	6
6 Sparse Unbalanced Entropic OT	7
6.1 目标函数	7
6.2 为什么 KL 允许状态消失或新生	7
6.3 推荐优化方法	7
7 代码落地补充操作	8
7.1 新增 network method: light_cci	8
7.2 新增 HKS 表征函数	9
7.3 新增 PIJ method: laplacian_sr_sparse_unbalanced_ot	9
7.4 配置项建议	9
7.5 profile-only 必须补	10
7.6 输出文件建议	10
8 实现时需要避免的坑	10
9 最小验证实验	11

10 建议命令形态	11
10.1 先 profile-only	11
10.2 再正式跑	11
11 结论	12

1 总体思路

现在的问题不是“有没有 OT”，而是原来的表达特征太大，且完整 source-target 成本矩阵的规模是二次的。新的路线把任务拆成四步：

1. **先建轻量图**。每个 organ、时间点、层级只构造 light_cci 图：基因层保留跨细胞基因调控，细胞层用 CCI，domain 层用 DDI 或聚合后的 CCI/DDI；暂时不混入原来的 GRN 大特征。
2. **再做图结构表征**。对每个时间点内部图做 Laplacian-HKS，把每个节点压缩为低维结构特征。
3. **用预成本筛候选**。用 Laplacian-HKS、SR/发育状态量、空间坐标构造预成本 B_{ij} ，只保留最可能的跨时间候选边 Ω 。
4. **在稀疏候选上做 OT**。用辅助图最短路得到最终转移成本 C_{ij} ，再用 sparse unbalanced entropic OT 优化得到 P_{ij} 。

light_cci 图 $\rightarrow$ Laplacian-HKS 特征 $\rightarrow$ KNN 候选边 $\rightarrow$ 辅助图路径成本 $\rightarrow$ sparse unbalanced OT $\rightarrow P_{ij}$.

2 符号系统

2.1 样本组与节点集合

$$g = (c', t, \ell).$$

这里， g 表示一个样本组； c' 表示器官或组织，例如 heart； t 表示发育时间点； ℓ 表示层级，例如 cell、spot、domain 或 gene-level layer。

对任意 $g = (c', t, \ell)$ ，定义：

$$V_{\ell,t}^{(c')} = \{\text{该器官、该时间点、该层级的所有节点}\}.$$

这里， $V_{\ell,t}^{(c')}$ 是节点集合；如果 ℓ 是 cell/spot 层，节点就是 cell/spot；如果 ℓ 是 domain 层，节点就是 domain；如果 ℓ 是基因层，节点可以是跨细胞调控网络里的基因或 gene-state unit。

2.2 LR pair 与 COMMOT 通讯矩阵

对某一个 ligand-receptor pair q ，COMMOT 可以给出一个稀疏通讯矩阵：

$$M_{\ell,t}^{(q)} \in \mathbb{R}_+^{|V_{\ell,t}^{(c')}| \times |V_{\ell,t}^{(c')}|}.$$

这里， $M_{\ell,t}^{(q)}$ 表示在层级 ℓ 、时间 t 上，第 q 个 LR pair 的通讯矩阵；矩阵元素 $M_{uv}^{(q)}$ 表示节点 u 通过该 LR pair 对节点 v 的通讯强度。

所有 LR pair 可以聚合成一张 light_cci 邻接矩阵：

$$A_{\ell,t} = \sum_{q \in Q_{\ell,t}} M_{\ell,t}^{(q)}.$$

这里， $Q_{\ell,t}$ 表示该层级、该时间点可用的 LR pair 集合； $A_{\ell,t}$ 是聚合后的有向通讯强度矩阵。

3 Laplacian-HKS 节点特征

3.1 从 light_cci 邻接矩阵到 normalized Laplacian

普通 graph Laplacian 通常需要无向图。因此先把有向通讯矩阵对称化：

$$\tilde{A}_{\ell,t} = \frac{1}{2} (A_{\ell,t} + A_{\ell,t}^\top).$$

这里， $\tilde{A}_{\ell,t}$ 是对称后的邻接矩阵； $A_{\ell,t}^\top$ 表示 $A_{\ell,t}$ 的转置。
定义度矩阵：

$$D_{\ell,t} = \text{diag}(\tilde{A}_{\ell,t} \mathbf{1}).$$

这里， $D_{\ell,t}$ 是对角矩阵； $\mathbf{1}$ 是全 1 向量； $\tilde{A}_{\ell,t} \mathbf{1}$ 得到每个节点的加权重。
normalized Laplacian 定义为：

$$L_{\ell,t} = I - D_{\ell,t}^{-1/2} \tilde{A}_{\ell,t} D_{\ell,t}^{-1/2}.$$

这里， $L_{\ell,t}$ 是 normalized Laplacian； I 是单位矩阵； $D_{\ell,t}^{-1/2}$ 表示对角元素开方后取倒数。

3.2 谱分解与 HKS

对 $L_{\ell,t}$ 做特征分解：

$$L_{\ell,t} \phi_m = \lambda_m \phi_m, \quad m = 1, 2, \dots, K.$$

这里， m 是第几个谱分量的编号； K 是保留的谱分量数量； λ_m 是第 m 个特征值； ϕ_m 是对应的特征向量； $\phi_m(i)$ 表示该特征向量在节点 i 上的取值。

直接使用 eigenvector 坐标会有符号翻转和旋转不唯一问题。因此主方案使用 heat kernel signature：

$$z_{i,h}^{\text{lap}} = \sum_{m=1}^K \exp(-\tau_h \lambda_m) \phi_m(i)^2, \quad h = 1, 2, \dots, H.$$

这里， $z_{i,h}^{\text{lap}}$ 是节点 i 在第 h 个扩散尺度上的 HKS 值； h 是扩散尺度编号； H 是扩散尺度数量； τ_h 是第 h 个 heat diffusion time； $\phi_m(i)^2$ 用平方消除 eigenvector 的符号任意性。

最终节点特征为：

$$z_i^{\text{lap}} = [z_{i,1}^{\text{lap}}, z_{i,2}^{\text{lap}}, \dots, z_{i,H}^{\text{lap}}].$$

默认超参建议。第一版可以设 $K = 32$, $H = 6$ 。 τ_h 建议按 Laplacian 特征值范围自动取 log-spaced scales，例如从 $1/(\lambda_K + \epsilon)$ 到 $1/(\lambda_2 + \epsilon)$ 取 H 个对数间隔点。

4 source-target 预成本与候选集合

4.1 两个时间点的图与节点特征

对一个时间对 (t_0, t_1) , 定义 source 图和 target 图:

$$G_0 = (S, E_0, w_0), \quad G_1 = (T, E_1, w_1).$$

这里, G_0 是 source 时间 t_0 的 light_cci 图; G_1 是 target 时间 t_1 的 light_cci 图; S 是 source 节点集合; T 是 target 节点集合; E_0 和 E_1 分别是两个时间点内部的 CCI/DDI 边集合; w_0 和 w_1 是内部边权函数。

对 $i \in S$ 和 $j \in T$, 定义:

- z_i : source 节点 i 的 Laplacian-HKS 特征; z'_j : target 节点 j 的 Laplacian-HKS 特征。
- q_i : source 节点 i 的 SR 或发育状态量; q'_j : target 节点 j 的 SR 或发育状态量。
- r_i : source 节点 i 的空间坐标; r'_j : target 节点 j 的空间坐标。这里使用 r , 避免和表达矩阵 X 混淆。
- a_i : source 节点 i 的给定质量; b_j : target 节点 j 的给定质量。

4.2 预成本 B_{ij}

候选筛选只需要一个快速预成本, 不需要一开始就计算完整的最终 OT 成本:

$$B_{ij} = \frac{\lambda_z d_z(z_i, z'_j) + \lambda_q d_q(q_i, q'_j) + \lambda_r d_r(r_i, r'_j)}{\lambda_z + \lambda_q + \lambda_r}.$$

这里, B_{ij} 是从 source 节点 i 到 target 节点 j 的预成本; d_z 是 HKS 特征距离; d_q 是 SR/发育状态距离; d_r 是空间距离; $\lambda_z, \lambda_q, \lambda_r$ 是三个距离分量的非负权重。

重要解释。 B_{ij} 越小, 说明 i 和 j 越相似, 越应该优先保留。 B_{ij} 只用于筛候选边, 不直接作为最终 OT 成本重复加入目标函数。

4.3 候选集合 Ω

对每个 source 节点 i , 保留 B_{ij} 最小的 K_s 个 target 节点; 对每个 target 节点 j , 保留 B_{ij} 最小的 K_t 个 source 节点。候选集合写成:

$$\begin{aligned} \Omega &= \Omega_{S \rightarrow T} \cup \Omega_{T \leftarrow S}. \\ \Omega_{S \rightarrow T} &= \left\{ (i, j) : j \in \text{TopK}_{K_s}^{\text{small}}(B_{i,:}) \right\}, \\ \Omega_{T \leftarrow S} &= \left\{ (i, j) : i \in \text{TopK}_{K_t}^{\text{small}}(B_{:,j}) \right\}. \end{aligned}$$

这里, Ω 是跨时间候选边集合; K_s 是每个 source 保留多少个 target 候选; K_t 是每个 target 反向保留多少个 source 候选。

覆盖诊断必须输出。 每个 source 是否至少有 K_s 个候选; 每个 target 是否至少有若干候选流入; 候选边覆盖了多少连通分量; $K_s = 20, 50, 100$ 时 EI/DI 是否稳定。这些诊断比单独报告一个最终 EI 更重要。

5 辅助图与路径成本

5.1 辅助图定义

候选边只告诉我们哪些跨时间匹配值得考虑。为了让成本更符合网络流形，构造一个跨时间辅助图：

$$\mathcal{H} = (V_H, E_H, w_H).$$

$$V_H = S \cup T, \quad E_H = E_0 \cup E_1 \cup \Omega.$$

这里， $\mathcal{H}$ 是辅助图； V_H 是辅助图节点集合； E_H 是辅助图边集合； w_H 是辅助图边权函数。注意：节点和节点取并集，边和边取并集，不是节点集和边集混在一起。

5.2 内部边代价

对内部 CCI/DDI 边 $e = (u, v)$ ，先把通讯强度转成路径代价。假设 $s_t(u, v)$ 是时间 t 上边 $u \rightarrow v$ 的聚合 CCI/DDI 强度，可以定义：

$$\tilde{s}_t(u, v) = \log(1 + s_t(u, v)),$$

$$p_t(v | u) = \frac{\tilde{s}_t(u, v) + \epsilon_c}{\sum_{v': (u, v') \in E_t} \tilde{s}_t(u, v') + \epsilon_c |N_t^+(u)|},$$

$$c_t(u, v) = -\log p_t(v | u).$$

这里， $\tilde{s}_t$ 是 $\log$ 压缩后的通讯强度； $p_t(v | u)$ 是从 u 到 v 的归一化通讯概率； $N_t^+(u)$ 是节点 u 的出邻居集合； ϵ_c 是防止除零的小常数； $c_t(u, v)$ 是内部边移动代价。强通讯边对应较小代价，弱通讯边对应较大代价。

辅助图边权函数为：

$$w_H(e) = \begin{cases} \gamma_0 c_0(e), & e \in E_0, \\ \gamma_1 c_1(e), & e \in E_1, \\ B_{ij}, & e = (i, j) \in \Omega. \end{cases}$$

这里， $c_0(e)$ 是 **source** 内部边代价； $c_1(e)$ 是 **target** 内部边代价； γ_0 和 γ_1 是控制 **source/target** 内部移动代价强度的超参； B_{ij} 是跨时间候选边的预成本。

5.3 最终 OT 成本

最终成本只保留辅助图路径距离：

$$C_{ij} = \text{normalize}(\text{dist}_{\mathcal{H}}(i, j)), \quad (i, j) \in \Omega_{\text{final}}.$$

这里， C_{ij} 是最终进入 OT 的成本； $\text{dist}_{\mathcal{H}}(i, j)$ 是在辅助图 $\mathcal{H}$ 上从 **source** 节点 i 到 **target** 节点 j 的最短路径距离； Ω_{final} 是最终保留的稀疏候选集合。

避免重复建模。 Laplacian-HKS 与 SR/空间信息先用于构造 B_{ij} 并筛选 Ω 。最终 OT 只使用 $C_{ij} = \text{normalize}(\text{dist}_{\mathcal{H}}(i, j))$ 。不要再把 d_z, d_q, d_r 又一次线性加进 C_{ij} ，否则 CCI 信息会被重复加权。

6 Sparse Unbalanced Entropic OT

6.1 目标函数

在稀疏候选集合上求转移矩阵：

$$P \in \mathbb{R}_+^{|S| \times |T|}, \quad \text{supp}(P) \subseteq \Omega_{\text{final}}.$$

这里， P_{ij} 表示从 **source** 节点 i 到 **target** 节点 j 的转移强度； $\text{supp}(P)$ 表示 P 的非零位置集合。主优化目标写成：

$$\min_{P \geq 0, \text{supp}(P) \subseteq \Omega_{\text{final}}} \sum_{(i,j) \in \Omega_{\text{final}}} P_{ij} C_{ij} + \varepsilon \sum_{(i,j) \in \Omega_{\text{final}}} P_{ij} (\log P_{ij} - 1) + \tau_s \text{KL}(P\mathbf{1} \| a) + \tau_t \text{KL}(P^\top \mathbf{1} \| b).$$

这里，第一项让大的 P_{ij} 尽量落在小的 C_{ij} 上；第二项是 **entropy regularization**，使解更平滑、更可优化； ε 控制平滑强度；第三项和第四项是 **source/target** 质量守恒的软约束； τ_s 控制 **source** 质量守恒强度； τ_t 控制 **target** 质量守恒强度。

KL 项定义为：

$$\begin{aligned} \text{KL}(r \| a) &= \sum_i \left[r_i \log \frac{r_i}{a_i} - r_i + a_i \right], \quad r = P\mathbf{1}. \\ \text{KL}(c \| b) &= \sum_j \left[c_j \log \frac{c_j}{b_j} - c_j + b_j \right], \quad c = P^\top \mathbf{1}. \end{aligned}$$

这里， r_i 是 **source** 节点 i 实际送出的质量； c_j 是 **target** 节点 j 实际收到的质量。

6.2 为什么 KL 允许状态消失或新生

普通 **balanced OT** 强制 $P\mathbf{1} = a$ 和 $P^\top \mathbf{1} = b$ 。这意味着每个 **source** 必须把质量全部送出，每个 **target** 必须被旧状态完全解释。发育过程中这太硬，因为状态可能消失、新生、扩增或采样变化。

unbalanced OT 把硬等式换成 KL 惩罚。如果某个 **source** 节点 i 没有合理 **target**，模型可以让：

$$r_i = \sum_j P_{ij} < a_i.$$

这表示它没有把全部质量送出，数学上对应“状态部分消失”或“没有清晰后继”。如果某个 **target** 节点 j 没有合理 **source**，模型可以让：

$$c_j = \sum_i P_{ij} < b_j.$$

这表示它没有完全由上一时间点解释，数学上对应“状态新生”或“前驱不清晰”。偏离越大，KL 惩罚越大，因此模型不是任意丢质量，而是在“强行匹配的路径成本”和“质量不守恒惩罚”之间做权衡。

6.3 推荐优化方法

目标函数使用 **sparse generalized Sinkhorn** 优化。先定义稀疏 Gibbs kernel：

$$K_{ij} = \exp\left(-\frac{C_{ij}}{\varepsilon}\right), \quad (i, j) \in \Omega_{\text{final}}.$$

只在候选边上存储 K_{ij} 。迭代更新：

$$u = \left(\frac{a}{Kv + \delta}\right)^{\rho_s}, \quad v = \left(\frac{b}{K^\top u + \delta}\right)^{\rho_t},$$

$$\rho_s = \frac{\tau_s}{\tau_s + \varepsilon}, \quad \rho_t = \frac{\tau_t}{\tau_t + \varepsilon}.$$

这里， u 和 v 是 Sinkhorn scaling vectors； K 是稀疏 kernel； δ 是数值稳定项； ρ_s 和 ρ_t 是 unbalanced 更新指数。最后得到：

$$P_{ij} = u_i K_{ij} v_j, \quad (i, j) \in \Omega_{\text{final}}.$$

为了兼容当前 EI 代码，如果后续需要行随机 PIJ，可以导出：

$$\hat{P}_{ij} = \frac{P_{ij}}{\sum_{j'} P_{ij'} + \delta}.$$

同时保留未归一化的 P ，用于质量消失/新生诊断。

7 代码落地补充操作

7.1 新增 network method: light_cci

新增文件：

```
mignet_ce/networks/light_cci.py
```

建议直接复用当前 clean_expression_cci_mix 的路径检查和 CCI 图构造逻辑，但不要构建大 expression block，也不要构建 native CCI feature matrix。输出的 NetworkContext 中：

- lower_graphs 和 upper_graphs 保留 light_cci 图；
- lower_mats 和 upper_mats 可以先放 np.zeros((n_units, 0)) 占位；
- feature_alignment_space="native_units"；
- metadata["feature_source"]="light_cci_graph_only"；
- metadata["uses_grn"]=False；
- metadata["uses_cci"]=True；
- metadata["uses_expression_feature"]=False；
- 表达数据只允许用于读取节点、坐标、可选 LR 表达过滤，不再作为高维特征进入 PIJ。

需要修改：

```
mignet_ce/networks/registry.py
mignet_ce/config.py
```

把 light_cci 加入 NETWORK_BUILDERS 和 NETWORK_METHODS。

7.2 新增 HKS 表征函数

当前代码已有普通 Laplacian embedding。需要新增 HKS，而不是覆盖旧方法：

```
mignet_ce/embeddings.py
- graph_to_adjacency(...)
- laplacian_hks_features(...)
- layer_graph_laplacian_hks_features(...)
```

关键要求：

- 支持 `n_eigen=32`、`n_scales=6`；
- 支持 `tau_strategy="auto"`；
- 对孤立点返回零向量或小常数向量；
- 输出每个节点的 H 维 HKS 特征；
- 导出 `eigen convergence`、实际保留特征数、耗时和内存诊断。

7.3 新增 PIJ method: laplacian_sr_sparse_unbalanced_ot

新增文件：

```
mignet_ce/pij/laplacian_sr_sparse_unbalanced_ot.py
mignet_ce/transition/geodesic_cost.py
mignet_ce/transition/sparse_unbalanced_ot.py
```

主流程：

1. 从 `context.lower_graphs[t]` 和 `context.upper_graphs[t]` 计算 HKS 特征。
2. 读取 development feature table 中的 SR 或发育状态量。
3. 从 `context.lower_coords_by_time` 和 `context.upper_coords_by_time` 读取坐标。
4. 构造标准化预成本 B_{ij} ，用 approximate nearest neighbors 选候选 Ω 。
5. 从 light_cci 图内部边构造辅助图 $\mathcal{H}$ ，内部边数按 top-r per node 截断。
6. 对每个 source 的候选 target 做局部最短路，得到稀疏 C_{ij} 。
7. 在 sparse C 上运行 generalized Sinkhorn。
8. 导出 sparse PIJ、行归一化 PIJ、候选覆盖诊断和质量诊断。

7.4 配置项建议

在 `TemporalRunConfig` 中增加：

```
hks_eigen_components: int = 32
hks_scales: int = 6
hks_tau_strategy: str = "auto"
sparse_candidate_k_source: int = 50
sparse_candidate_k_target: int = 20
sparse_use_reverse_knn: bool = True
aux_internal_topk_per_node: int = 50
aux_gamma_source: float = 1.0
aux_gamma_target: float = 1.0
sparse_ot_epsilon: float = 0.05
sparse_ot_mass_source: float = 1.0
sparse_ot_mass_target: float = 1.0
sparse_ot_max_iter: int = 200
```

```
sparse_ot_tol: float = 1e-6
dijkstra_batch_size: int = 256
profile_only: bool = False
```

7.5 profile-only 必须补

新增脚本或运行模式：

```
scripts/profile_light_cci_pij.py
# 或者在 run_mignet_vertical_ablation.py 里加 --profile-only
```

profile-only 不真正跑完整 PIJ，只输出规模估计：

- 每个 organ/time/layer 的节点数；
- LR manifest 文件数、LR 文件总大小、总 nnz、最大 nnz；
- 预计 CCI inter edge 数量；
- Laplacian adjacency nnz、预计 eigensolver 规模；
- 候选边 $|\Omega| \approx |S|K_s + |T|K_t$ ；
- 辅助图边数 $|E_H|$ ；
- sparse OT memory: $\text{nnz} * (\text{cost} + \text{row} + \text{col} + \text{kernel} + P)$ ；
- dense OT memory 对照: $|S| * |T| * 8 * \text{multiplier}$ ；
- 预估能否上 GPU，或是否应该 CPU sparse。

7.6 输出文件建议

每个 organ、层级对、时间对建议输出：

```
profile.json
units_source.csv
units_target.csv
hks_source.npy
hks_target.npy
candidate_edges.parquet
aux_graph_edges.parquet
geodesic_cost_sparse.npz
pij_transport_sparse.npz
pij_row_normalized_sparse.npz
mass_diagnostics.csv
ot_convergence.json
```

其中 mass_diagnostics.csv 至少包含 source 实际送出质量 r_i 、原质量 a_i 、target 实际收到质量 c_j 、目标质量 b_j ，用于解释哪些状态可能消失或新生。

8 实现时需要避免的坑

1. 不要生成完整 $|S| \times |T|$ 成本矩阵。候选筛选、路径成本、OT 都应该保持稀疏。
2. 不要把 **overlap/domain** 映射用于主 PIJ。profile 可以检查 coverage，但不能把跨层映射放进 lower 层 PIJ 成本。
3. 不要让 B_{ij} 和 C_{ij} 双重计分。 B_{ij} 用来建候选和跨时间边；最终 OT 成本用路径距离 C_{ij} 。
4. 不要无限保留 CCI 内部边。每个节点保留 top-r 内部边，避免辅助图变成高密度图。

5. 保留 **raw transport**。行归一化 PIJ 可以给 EI 使用，但 raw P 对解释 unbalanced mass 非常重要。

9 最小验证实验

实验	目的
profile-only	在真正跑之前估算 LR 文件、nnz、候选边、辅助图边、稀疏 OT 内存。
K sensitivity	比较 $K_s = 20, 50, 100$ 下 EI/DI 是否稳定，防止 KNN 删掉真实对应。
without auxiliary edges	只用跨时间候选边做 OT，检查内部 CCI 路径是否真的有贡献。
balanced vs unbalanced	比较 τ_s, τ_t 很大时和正常 unbalanced 时的结果，检查质量松弛是否影响结论。
randomized CCI control	打乱 CCI 边权或保持度分布随机化，确认结果不是由节点数或边密度本身驱动。

10 建议命令形态

10.1 先 profile-only

```
python scripts/profile_light_cci_pij.py \
--data-root public/datasets/Mouse-embryo/E1S1_domain_factory \
--output-root output/profile_light_cci \
--organs heart \
--time-points 11.5 12.5 13.5 14.5 \
--network-method light_cci \
--pij-method laplacian_sr_sparse_unbalanced_ot \
--candidate-k-source 50 \
--candidate-k-target 20 \
--aux-internal-topk-per-node 50 \
--profile-only \
--progress
```

10.2 再正式跑

```
python scripts/run_mignet_vertical_ablation.py \
--data-root public/datasets/Mouse-embryo/E1S1_domain_factory \
--output-root output/light_cci_laplacian_sr_sparse_uot \
--organs heart \
--time-points 11.5 12.5 13.5 14.5 \
--network-method light_cci \
--pij-method laplacian_sr_sparse_unbalanced_ot \
--development-feature-root public/datasets/Mouse-embryo/E1S1_domain_factory/development_features \
--export-pij \
--progress \
--progress-log output/light_cci_laplacian_sr_sparse_uot/progress.jsonl
```

11 结论

这个版本的重点不是把所有信息都堆进成本函数，而是把每一种信息放到合适的位置：**light_cci** 图提供低成本网络骨架；**Laplacian-HKS** 提供稳定节点结构表征；**SR**/空间信息辅助筛候选；辅助图最短路把候选约束转化为网络路径成本；**sparse unbalanced OT** 在稀疏候选上优化 **PIJ**，并允许状态消失或新生。

主方案： light_cci + Laplacian-HKS candidate + auxiliary geodesic cost + sparse unbalanced Sinkhorn